Algorithms homework3

Created	@December 7, 2025 8:35 AM

Варіант 2. Максимальне паросполучення з забороненими вершинами.

Дано неорієнтований граф G = (V, E) та підмножина вершин B ⊆ V , які заборонено включати в паросполучення. Потрібно знайти паросполучення максимальної
потужності (максимальну кількість ребер), що не інцидентні жодній вершині з B.
Вхідні дані: n — кількість вершин, список заборонених вершин B, кількість ребер,
список ребер.
Вихідні дані: множина ребер максимального паросполучення та його розмір.

Алгоритм реалізований на мові програмування C++, порівняння виконане на Python

Як запускати код на C++ (не перевіряла на Mac чи спрацює, але на вінді працює)

```
g++ -std=c++17 -O2 max_maching_forbidden.cpp -o max_matching_forbidde
n.exe
.\max_matching_forbidden.exe test1.csv
```

Як запускати код на Python (не перевіряла на Mac чи спрацює, але на вінді працює)

```
python max_matching_forbidden.py test1.csv
```

Переформульована мета алгоритму

Знайти **максимальне паросполучення** — найбільшу за кількістю множину ребер, таку що:

- ☐ жодні два ребра не мають спільної вершини
- ☐ жодне ребро не інцидентне жодній вершині з множини **B**

Формат вхідних даних

Вхідні дані приходять у вигляді csv файлу, приклад вхідного файлу:

```
n,k,m
4,0,3
forbidden_vertices

u,v
1,2
2,3
3,4
```

```
n,k,m
4,0,3
```

означає:

- `n = 4` — кількість вершин графа, у цьому випадку у графі **4 вершини**
- `k = 0` — кількість заборонених вершин, у цьому випадку **немає заборонених вершин**
- `m = 3` — кількість ребер, у даному прикладі **3 ребра**

Після рядка `forbidden_vertices` йдуть заборонені вершини, наприклад:

```
forbidden_vertices
3
```

3 - заборонена вершина

```
u,v
1,2
2,3
3,4
```

Це означає, що в графі є ребра:

- між 1 і 2
- між 2 і 3
- між 3 і 4

Формат вихідних даних

```
2
1 2
4 5
```

- перший рядок - розмір максимального паросполучення
- потім - список ребер, що входять у нього

Загальна логіка роботи алгоритму

Крок 1: Обробка заборонених вершин

Ідея:

Вершини з множини **B** не можуть входити до паросполучення.

Реалізація:

Усі ребра, інцидентні забороненим вершинам, **не додаються** до графа під час зчитування.

Результат:

Задача зводиться до:

> знайти максимальне паросполучення в графі без заборонених вершин

Крок 2: Алгоритм Едмондса (Blossom)

Вся специфіка варіанту:

> «максимальне паросполучення з забороненими вершинами»

реалізується дуже просто:

- вершини з `B` не видаляються з `V`,
- але **всі ребра з ними просто не додаються** у `adj`.

Тоді:

- алгоритм Едмондса працює на графі `G' = (V, E')`,
- де `E'` — лише ребра, що не чіпають заборонені вершини.

Важливий момент:

> після такого фільтрування граф G' усе ще може бути дуже складним:
>
> з blossom-циклами, дивними компонентами і т.п.

Отже:

- потрібен алгоритм, який **гарантовано** дає максимальне паросполучення в **загальному** графі,
- **без додаткових структурних припущень**.

Ці вимоги ідеально задовольняє Blossom.

Використаний алгоритм:

Алгоритм Едмондса (Edmonds' Blossom Algorithm)

— працює для довільних неорієнтованих графів

— коректно обробляє непарні цикли

Я використовую його, тому що алгоритм Blossom:

- дає **поліноміальний час** для maximum matching в **загальному графі**;
- вміє коректно обробляти **непарні цикли (blossoms)**:
  - знаходить «погане» ребро між двома парними вершинами,
  - виявляє непарний цикл,
  - **стискає** його в одну надвершину (через `base[]`),
  - продовжує BFS у стислому графі,
  - і, якщо знаходить augmenting path, потім «розгортає» його назад.

Основна ідея алгоритму

- Будуємо **чергувальні шляхи**
- Якщо знайдено шлях поповнення → паросполучення збільшується
- Якщо знайдено непарний цикл → виконується згортання (*blossom contraction*)
- Процес повторюється, поки можливе покращення

Основні структури даних

Змінна	Призначення
`match[v]`	напарник вершини `v` або `-1`, якщо вільна
`parent[v]`	батьківська вершина в BFS
`base[v]`	базова вершина blossom
`used[v]`	чи була вершина оброблена
`blossom[v]`	чи належить вершина поточному blossom

Змінна	Призначення
`q`	черга BFS

Побудова “чергувального” дерева

- **Чергувальне дерево** — це як “ліс можливих маршрутів” від кореня, побудований за правилами чергування ребер.
- **Доповнювальний шлях** — це один із цих маршрутів,

  який з’єднує **вільну вершину з вільною вершиною**.

Ініціалізація BFS

- ☐ стартуємо з вільної вершини
- ☐ вона є коренем чергувального дерева
- ☐ скидаємо `parent`, `base`, `used`

Побудова шляхів

Ребра чергуються:

```
вільне → парне → вільне → парне → ...
```

Випадки під час BFS

Ситуація 1: знайдено вільну вершину

- знайдено **шлях поповнення**
- виконується "перевертання" пар
- розмір паросполучення збільшується

Ситуація 2: непарний цикл (Blossom)

Коли:

- знайдено ребро між вершинами одного рівня

Дія:

- шукаємо LCA (спільну базу)
- згортаємо цикл
- продовжуємо BFS

Повторення

- ☐ запускаємо BFS з кожної вільної вершини
- ☐ при успіху — оновлюємо matching
- ☐ якщо BFS не дає шляху — зупинка

Завершення

Алгоритм завершується, коли не існує augmenting path

Поточне паросполучення — максимальне

Формування результату

- проходимо по `match[]`
- формуємо список пар
- виводимо розмір і список ребер

Оцінка складності

```
O(V^3), де V — кількість вершин
```

Коректність щодо заборонених вершин

Вершини з B ніколи не з'являються у результаті

Всі інцидентні ребра видалені

Обмеження задачі дотримані автоматично

Опис реалізації

1. Загальна ідея програми

Програма складається з двох основних частин:

1. **Читання та підготовка графа** з CSV-файлу:
   - зчитуються `n, k, m`,
   - зчитуються заборонені вершини,
   - зчитуються ребра та **фільтруються** ті, які інцидентні забороненим вершинам,
   - будується список суміжності `adj`.
2. **Пошук максимального паросполучення** в отриманому графі:
   - використовується структура `EdmondsBlossom` з реалізацією алгоритму Едмондса,
   - виводиться розмір паросполучення і список пар вершин.

---

2. Структура `EdmondsBlossom`

```
struct EdmondsBlossom {
    int n;
    std::vector<std::vector<int>> g;
```

```
    std::vector<int> match;
    std::vector<int> parent;
    std::vector<int> base;
    std::vector<int> q;
    std::vector<bool> used;
    std::vector<bool> blossom;
    ...
};
```

2.1. Поля структури

- `n` — кількість вершин у графі.
- `g` — список суміжності: `g[v]` — вектор усіх сусідів вершини `v`.
- `match[v]` — для якої вершини `v` зараз спарена:
  - `match[v] = u`, якщо вершина `v` знаходиться в парі з `u`;
  - `match[v] = -1`, якщо вершина поки **вільна**.
- `parent[v]` — батьківська вершина `v` у дереві BFS (чергувальне дерево).
- `base[v]` — **базова вершина** для компоненти / blossom-а, до якого належить `v`.

  Коли blossom згортається, кілька вершин отримують один і той самий `base`.
- `q` — масив, який використовується як **черга BFS**:
  - `qh` — “голова” (індекс читання),
  - `qt` — “хвіст” (індекс запису).
- `used[v]` — чи вершина вже потрапила в чергу BFS (була відвідана як частина дерева).
- `blossom[v]` — тимчасова позначка: чи вершина належить поточному blossom, що згортається.

2.2. Конструктор

```
EdmondsBlossom(int n_, const std::vector<std::vector<int>>& adj)
    : n(n_), g(adj),
      match(n, -1),
      parent(n),
      base(n),
      q(n),
      used(n),
      blossom(n)
{}
```

- `g` ініціалізується копією `adj`.
- `match` ініціалізується вектором з `n` елементів, усе `1` (усі вершини вільні).
- `parent`, `base`, `q`, `used`, `blossom` — просто створюються потрібного розміру.

---

3. Функція `findLCA(int a, int b)`

```
int findLCA(int a, int b) {
    std::vector<bool> usedPath(n, false);
    while (true) {
        a = base[a];
        usedPath[a] = true;
        if (match[a] == -1) break;
        a = parent[match[a]];
    }
    while (true) {
        b = base[b];
        if (usedPath[b]) return b;
        if (match[b] == -1) break;
        b = parent[match[b]];
    }
    return -1;
}
```

Мета: знайти *Lowest Common Ancestor* (LCA) — спільну базову вершину blossom-а для двох вершин `a` і `b` в дереві чергувальних шляхів.

Логіка:

1. Йдемо вгору від `a`, піднімаючись по ланцюгу:
   - беремо `base[a]`,
   - позначаємо `usedPath[base[a]] = true`,
   - якщо `match[a] == -1` → досягли кореня, зупиняємось,
   - інакше переходимо до `parent[match[a]]`.

   Це відповідає руху вгору по чергувальному дереву через пари "спарена вершина → її батько".
2. Потім рухаємося вгору від `b` тим самим способом:
   - кожного разу дивимось на `base[b]`,
   - перша базова вершина, яку вже відвідали з боку `a` (`usedPath[b] == true`),

     — це і є спільна база blossom-а (LCA).

Це потрібно, коли ми знаходимо ребро між двома вершинами парного рівня в BFS і хочемо знайти весь непарний цикл.

---

4. Функція `markPath(int v, int b, int child)`

```
void markPath(int v, int b, int child) {
    while (base[v] != b) {
        blossom[base[v]] = blossom[base[match[v]]] = true;
        parent[v] = child;
        child = match[v];
        v = parent[match[v]];
    }
}
```

Мета: позначити вершини, що входять до blossom-а, по шляху від `v` до бази `b`.

- Ми рухаємося вгору від вершини `v`, доки її `base[v]` не стане рівною `b` (LCA).
- На кожному кроці:
  - позначаємо `blossom[base[v]] = true`;
  - позначаємо також `blossom[base[match[v]]] = true`, бо blossom включає і вершини-партнери;
  - оновлюємо `parent[v] = child`, щоб правильно відновити шляхи;
  - далі переходимо через парного партнера `match[v]` до `parent[match[v]]`.

Це дозволяє зібрати весь цикл blossom-а і помітити всі вершини, які потрібно "згорнути".

---

5. Функція `findAugmentingPath(int root)`

Це **серце алгоритму** — пошук шляху поповнення з кореня `root`.

5.1. Ініціалізація

```
std::fill(used.begin(), used.end(), false);
std::fill(parent.begin(), parent.end(), -1);
for (int i = 0; i < n; ++i) {
    base[i] = i;
}

int qh = 0, qt = 0;
q[qt++] = root;
used[root] = true;
```

- `used[v] = false` — ще нічого не в BFS.
- `parent[v] = -1` — немає батьків.
- `base[v] = v` — спочатку кожна вершина є своєю базою (немає blossom-ів).
- Черга `q`:
  - додаємо `root`,

- `used[root] = true`.

5.2. BFS по графу

```
while (qh < qt) {
    int v = q[qh++];

    for (int to : g[v]) {
        if (base[v] == base[to] || match[v] == to) continue;
        ...
    }
}
```

- Поки черга не порожня:
  - беремо вершину `v`,
  - переглядаємо всіх сусідів `to`.

Умова:

- `base[v] == base[to]` — ігноруємо ребро всередині одного й того ж blossom-а.
- `match[v] == to` — це “зворотне” парне ребро, його теж не розглядаємо як новий кандидат.

5.3. Випадок 1 — blossom (непарний цикл)

```
if (to == root || (match[to] != -1 && parent[match[to]] != -1)) {
    int blossomBase = findLCA(v, to);
    std::fill(blossom.begin(), blossom.end(), false);
    markPath(v, blossomBase, to);
    markPath(to, blossomBase, v);

    for (int i = 0; i < n; ++i) {
        if (blossom[base[i]]) {
            base[i] = blossomBase;
            if (!used[i]) {
                used[i] = true;
```

```
            q[qt++] = i;
        }
    }
  }
}
```

Ми потрапляємо сюди, якщо:

- `to == root` — знайшли цикл, що повертається у корінь,

  або

- `match[to] != -1 && parent[match[to]] != -1` — партнер `to` вже теж в дереві,

  тобто знов-таки маємо ребро між двома парними рівнями → odd cycle.

Тоді:

1. Знаходимо `blossomBase = findLCA(v, to)`.
2. Очищаємо `blossom[]`.
3. Викликаємо `markPath` з `v` і `to`, щоби позначити весь blossom.
4. Для всіх вершин:
   - якщо їх `base[i]` належить blossom-у, ставимо `base[i] = blossomBase`,
   - якщо вершина ще не використана в BFS (`!used[i]`), додаємо її в чергу.

🔍 **Нюанс:** blossom формально "згортається" у надвершину з базою `blossomBase`, але реалізація не змінює сам граф `g`, а тільки працює через масив `base[]`. Це економить перепаковку структури графа.

5.4. Випадок 2 — розширення дерева

```
else if (parent[to] == -1) {
    parent[to] = v;
    if (match[to] == -1) {
        // augment
    } else {
        int partner = match[to];
        if (!used[partner]) {
```

```
            used[partner] = true;
            q[qt++] = partner;
        }
    }
}
```

Якщо `to` ще не має батьків (`parent[to] == -1`):

- ставимо `parent[to] = v` — приєднуємо до дерева.

Далі два підвипадки:

5.4.1. Якщо `match[to] == -1`

Тобто `to` — **вільна вершина**:

```
int cur = to;
while (cur != -1) {
    int prev = parent[cur];
    int next = (prev == -1) ? -1 : match[prev];
    if (prev != -1) {
        match[cur] = prev;
        match[prev] = cur;
    }
    cur = next;
}
return true;
```

- Ми знайшли **augmenting path** від `root` до `to`.
- Йдемо назад по `parent[]`, на кожному кроці:
  - спаровуємо `cur` і `prev`,
  - рухаємося далі по старих сполученнях `next = match[prev]`.
- Після цього паросполучення стало більшим на 1.
- Повертаємо `true`.

5.4.2. Якщо `match[to] != -1`

```
int partner = match[to];
if (!used[partner]) {
    used[partner] = true;
    q[qt++] = partner;
}
```

- `to` вже в парі, але ми додаємо її напарника `partner` до BFS-черги:
  - так будується **чергувальне дерево**: вільне ребро → matched → вільне → matched.

---

6. Функція `maxMatching()`

```
int maxMatching() {
    int matchingSize = 0;
    for (int i = 0; i < n; ++i) {
        if (match[i] == -1) {
            if (findAugmentingPath(i)) {
                ++matchingSize;
            }
        }
    }
    return matchingSize;
}
```

- Перебираємо всі вершини.
- Якщо вершина `i` ще вільна — пробуємо побудувати augmenting path з неї.
- Якщо вдалося (функція повернула `true`), збільшуємо розмір паросполучення.
- В результаті маємо максимальний розмір.

---

7. Допоміжна функція `splitCSV`

```
static std::vector<std::string> splitCSV(const std::string& line) {
    ...
}
```

- Розбиває рядок за комами `,`.
- Обрізає пробіли/табуляції на початку та в кінці токенів.
- Повертає вектор токенів (у тому числі порожні, якщо між комами нічого немає).

Це дозволяє зручно парсити:

```
1,  2
```

і навіть:

```
1,   2,    3
```

---

8. Функція `main`

8.1. Налаштування введення

```
std::ios::sync_with_stdio(false);
std::cin.tie(nullptr);
```

- Прискорює введення/виведення (відключає синхронізацію з `stdio`).

```
std::istream* in = &std::cin;
std::ifstream file;
if (argc > 1) {
    file.open(argv[1]);
    ...
    in = &file;
}
```

- Якщо файл передано як аргумент командного рядка → читаємо з файлу.
- Інакше зчитуємо з `std::cin`.

8.2. Пропуск заголовка

```
if (!std::getline(*in, line)) {
    std::cerr << "Empty CSV input\n";
    return 0;
}
```

Перший рядок — це `n,k,m`, але в твоєму форматі перед ним є заголовок:

```
n,k,m
4,0,3
...
```

Тому перший `getline` просто "з'їдає" рядок `n,k,m`.

8.3. Зчитування `n,k,m`

```
if (!getline(*in, line)) {
    std::cerr << "Missing n,k,m values\n";
    return 0;
}
auto parts = splitCSV(line);
...
int n = stoi(parts[0]);
int k = stoi(parts[1]);
int m = stoi(parts[2]);
```

- Викликаємо `splitCSV`, беремо перші три числа.
- `m` у подальшій логіці **не використовується напряму** — це цікавий нюанс:
  - ми довіряємо тому, що вхід коректний,
  - кількість ребер рахуємо "як є" по рядках.

8.4. Заборонені вершини

```
if (!getline(*in, line)) {
    std::cerr << "Missing 'forbidden_vertices' label\n";
    return 0;
}

if (!getline(*in, line)) {
    std::cerr << "Missing forbidden vertices line\n";
    return 0;
}
parts = splitCSV(line);
std::vector<bool> forbidden(n, false);
...
```

- Спочатку очікуємо рядок `forbidden_vertices`.
- Потім рядок зі списком вершин (або порожній, якщо їх немає).
- Для кожного токена:
  - конвертуємо в `int` → 1-based номер вершини,
  - зменшуємо на 1 (робимо 0-based),
  - `forbidden[v] = true`.

Якщо кількість реально прочитаних заборонених вершин `countForbidden` не збігається з заявленим `k`, виводиться **warning**, але програма не падає — ще один нюанс.

8.5. Рядок заголовка ребер

```
if (!getline(*in, line)) {
    std::cerr << "Missing edges header 'u,v'\n";
    return 0;
}
```

- Читаємо рядок `u,v` — просто пропускаємо.

8.6. Зчитування ребер

```
std::vector<std::vector<int>> adj(n);
while (getline(*in, line)) {
    std::string trimmed = line;
    {
        size_t start = trimmed.find_first_not_of(" \t\r\n");
        size_t end   = trimmed.find_last_not_of(" \t\r\n");
        if (start == std::string::npos) {
            continue;
        }
        trimmed = trimmed.substr(start, end - start + 1);
    }

    parts = splitCSV(trimmed);
    if (parts.size() < 2) {
        std::cerr << "Warning: invalid edge line: " << trimmed << "\n";
        continue;
    }

    int u = stoi(parts[0]) - 1;
    int v = stoi(parts[1]) - 1;

    if (u < 0 || u >= n || v < 0 || v >= n) {
        std::cerr << "Skipping edge with invalid vertex index: " << trimmed <<
"\n";
        continue;
    }

    if (forbidden[u] || forbidden[v]) {
        continue;
    }

    adj[u].push_back(v);
```

```
    adj[v].push_back(u);
}
```

- Обрізаємо пробіли в рядку.
- Якщо рядок порожній → просто пропускаємо.
- Ділимо на два числа — `u`, `v`.
- Перетворюємо в 0-based індекси.
- Перевіряємо, що номери в межах `[0, n)`.
- **Головний момент:** якщо `forbidden[u] || forbidden[v]` → ребро не додаємо взагалі.
- Інакше додаємо в обидва списки суміжності.

Тут **заборонені вершини не видаляються з графа**, а просто стоять як ізольовані (або частково ізольовані, якщо були ребра тільки до них). Алгоритм просто ніколи їх не зможе використати в паросполученні, бо немає ребер.

---

9. Виклик алгоритму та формування відповіді

```
EdmondsBlossom blossom(n, adj);
int size = blossom.maxMatching();
```

- Створюємо об'єкт з нашим графом.
- Запускаємо максимальне паросполучення.
- `size` — кількість пар (ребер).

9.1. Відновлення пар

```
std::vector<std::pair<int,int>> edges;
std::vector<bool> used(n, false);
for (int v = 0; v < n; ++v) {
    int u = blossom.match[v];
    if (u != -1 && !used[v] && !used[u]) {
        used[v] = used[u] = true;
```

```
        int a = v + 1;
        int b = u + 1;
        if (a > b) std::swap(a, b);
        edges.emplace_back(a, b);
    }
}
```

- Перебираємо всі вершини `v`.
- Беремо `u = match[v]`.
- Якщо `u != -1` і обидві вершини ще не позначені як використані → це нова пара.
- Позначаємо `used[v] = used[u] = true`, щоб не дублювати (і не мати `(2,1)` і `(1,2)`).
- Конвертуємо назад в **1-based** номери (як у вхідних даних).
- `if (a > b) std::swap(a, b);` — впорядковуємо пару, щоб перший номер був меншим.

9.2. Сортування та вивід

```
sort(edges.begin(), edges.end());

std::cout << size << "\n";
for (auto [u, v] : edges) {
    std::cout << u << " " << v << "\n";
}
```

- Сортуємо список ребер лексикографічно (спочатку за `u`, потім за `v`).
- Друкуємо:
  - спочатку розмір паросполучення,
  - потім кожну пару в окремому рядку.

---

10. Цікаві нюанси реалізації

1. **Forbidden вершини обробляються “на вході”:**

- замість того, щоб ускладнювати сам алгоритм Едмондса, я просто **не додаю** ребра з забороненими вершинами;
- алгоритм узагалі не знає про існування обмеження — працює зі “звичайним” графом.

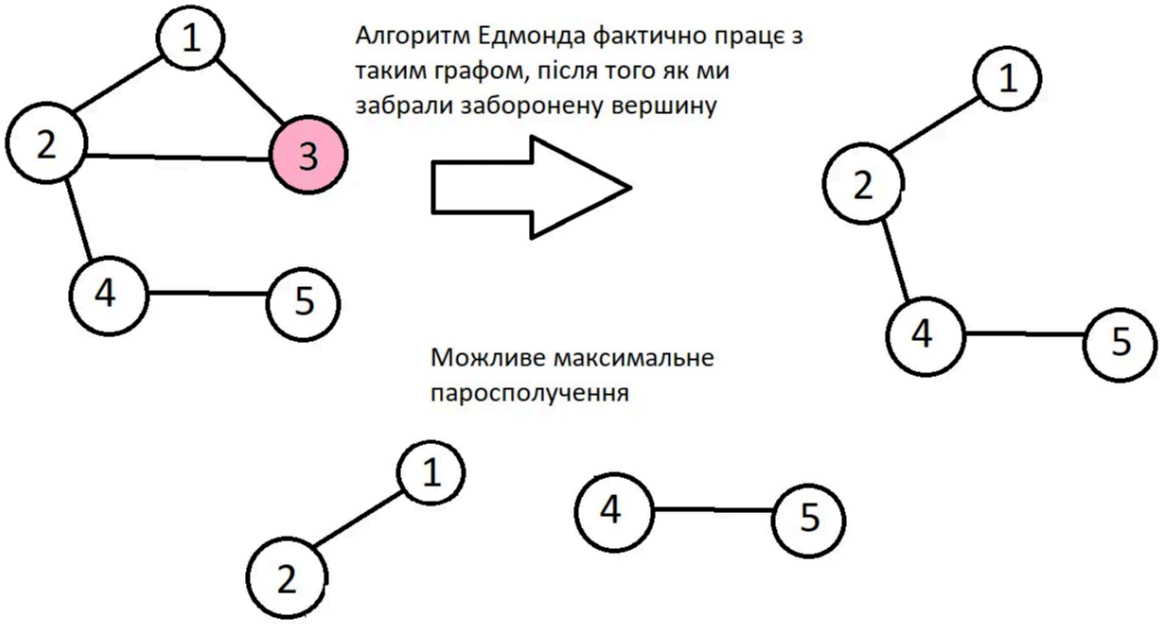
2. **Blossom не змінює сам граф:**
   - жодні вершини не видаляються й не додаються фізично,
   - усе відбувається через масив `base[]`, який дозволяє трактувати кілька вершин як одну.
3. **Черга BFS як масив + два індекси `qh`, `qt`:**
   - трохи швидше, ніж `std::queue`, але логіка та сама.
4. **Перехід між 0-based та 1-based:**
   - всередині реалізації — 0-based (масиви, індекси),
   - вхідні/вихідні дані — 1-based.
5. **Warning замість помилки для `k`:**
   - якщо кількість реально знайдених заборонених вершин не дорівнює `k`, програма не завершується,
   - це робить реалізацію стійкішою до дрібних помилок у файлі.

Приклади

Приклад №1

```
n,k,m
5,1,5
forbidden_vertices
3
u,v
1,2
1,3
```

```
2,3
2,4
4,5
```



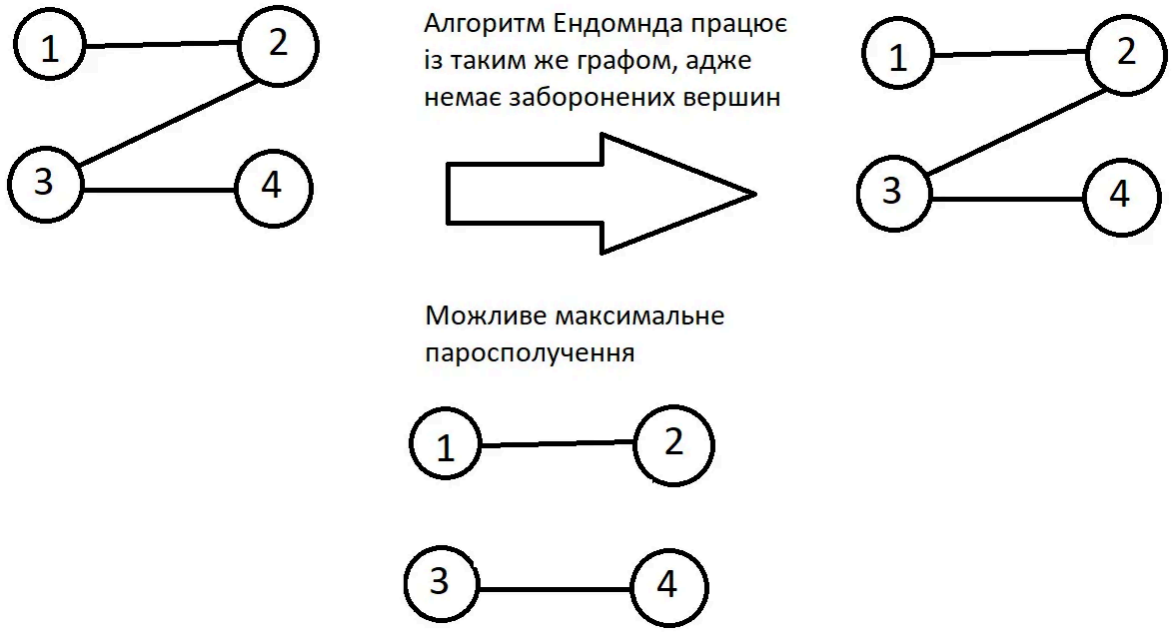
Що видає програми моя та програма на python із використанням функцій вбудованої бібліотеки networkx

```
PS D:\UCU\algorithms\homework3> .\max_matching_forbidden.exe test1.csv
2
1 2
4 5
PS D:\UCU\algorithms\homework3> python max_matching_forbidden.py test1.csv
2
1 2
4 5
```

Приклад №2

```
n,k,m
4,0,3
forbidden_vertices
```

```
u,v
1,2
2,3
3,4
```



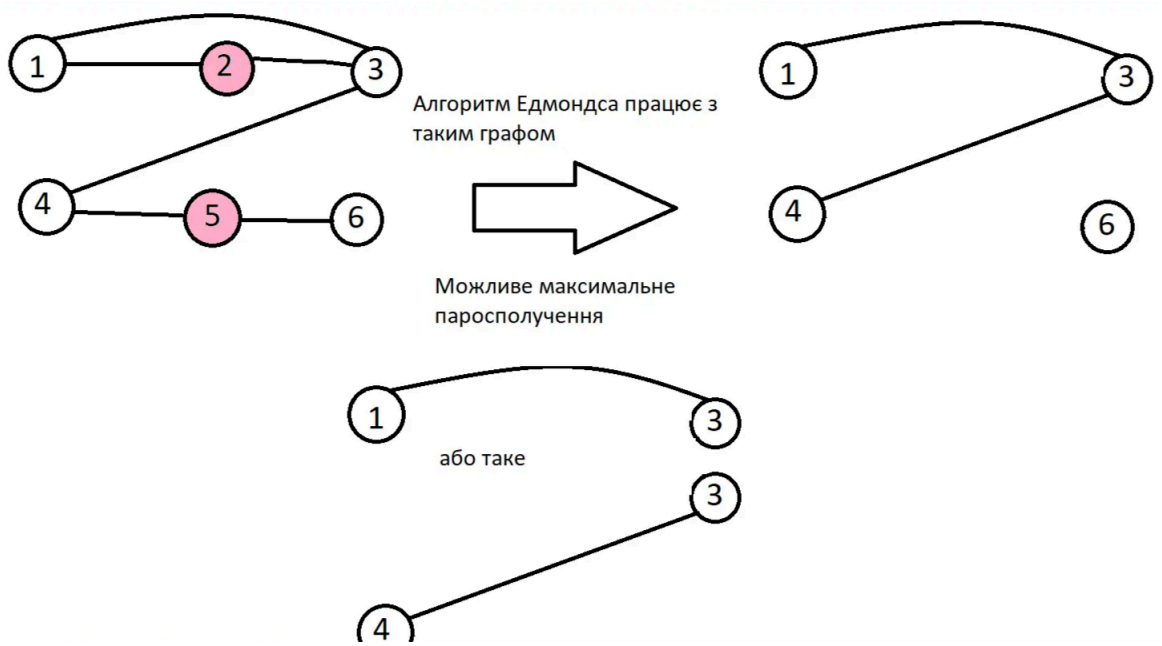
Помилка: правильно мало бути Едмондса

Що видає програми моя та програма на python із використанням функцій вбудованої бібліотеки networkx

```
PS D:\UCU\algorithms\homework3> .\max_matching_forbidden.exe test2.csv
2
1 2
3 4
PS D:\UCU\algorithms\homework3> python max_matching_forbidden.py test2.csv
2
1 2
3 4
```

Приклад №3

```
n,k,m
6,2,6
forbidden_vertices
2,5
u,v
1,2
1,3
2,3
3,4
4,5
5,6
```



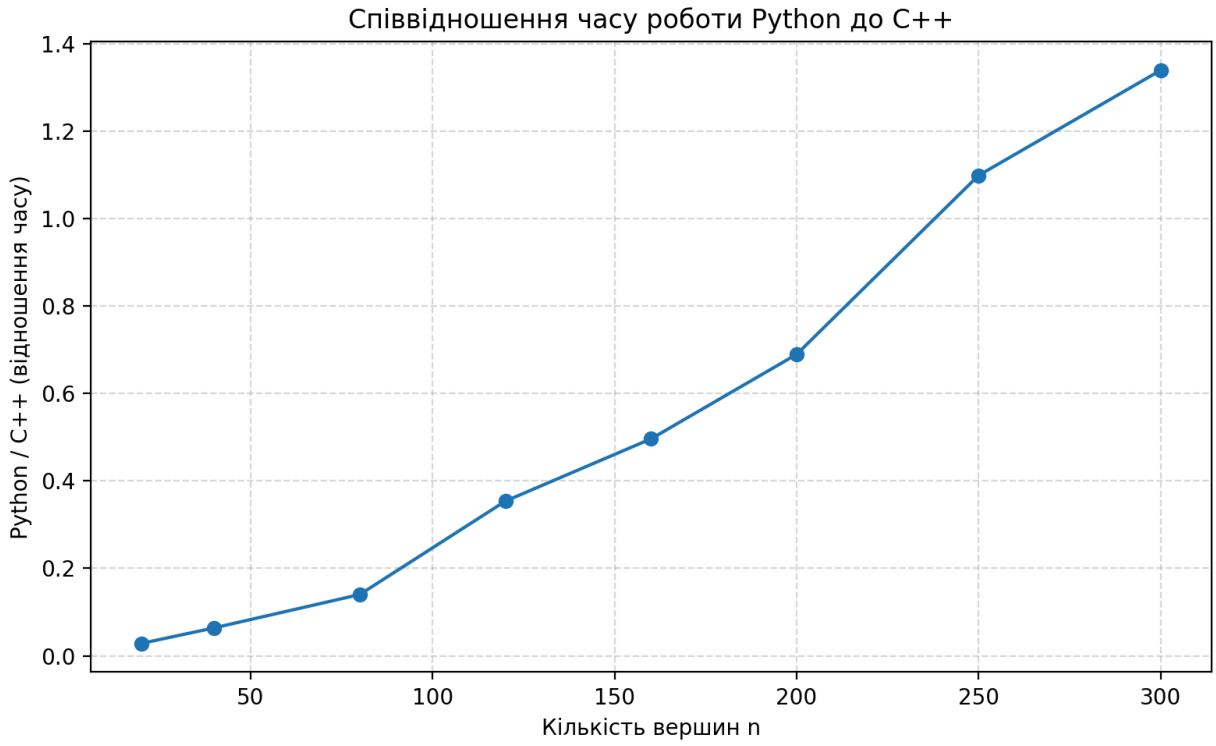
Що видає програми моя та програма на python із використанням функцій вбудованої бібліотеки networkx

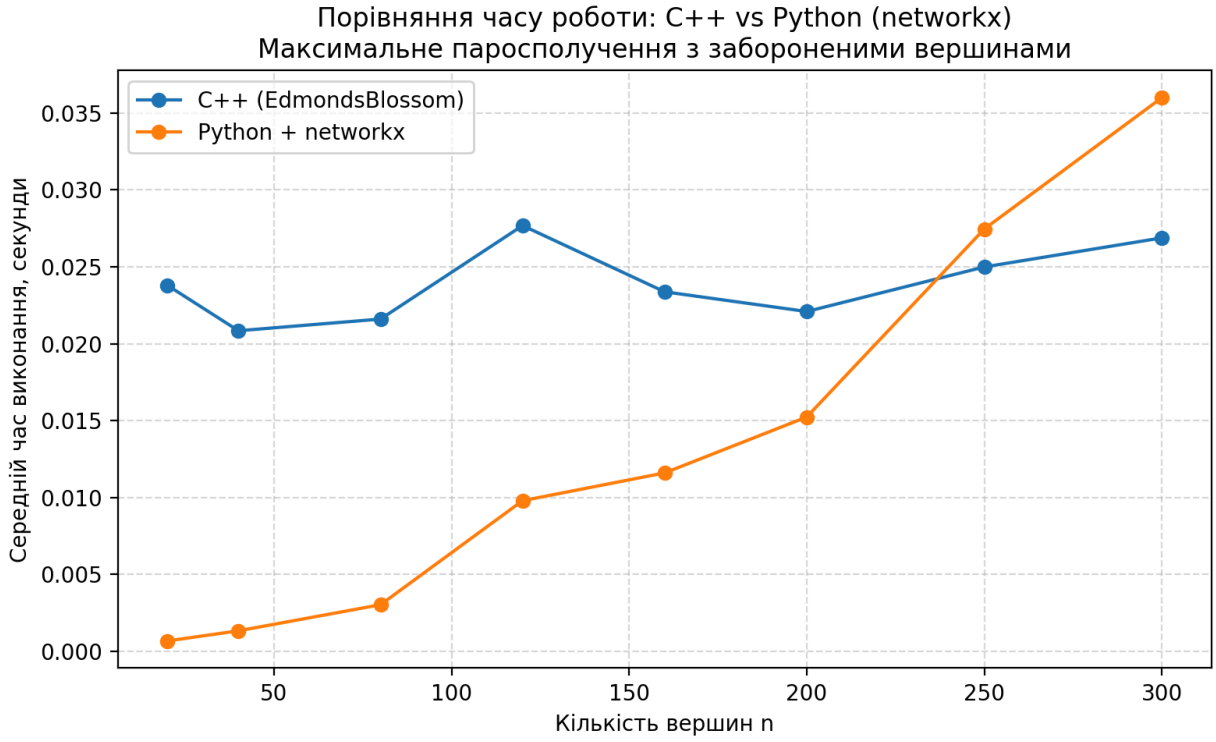
```
3 4
PS D:\UCU\algorithms\homework3> .\max_matching_forbidden.exe test3.csv
1
1 3
PS D:\UCU\algorithms\homework3> python max_matching_forbidden.py test3.csv
1
3 4
PS D:\UCU\algorithms\homework3>
```

Для деяких графів максимальне паросполучення **не є унікальним**, тобто існує кілька різних множин ребер однакового максимального розміру. Моя реалізація алгоритму Едмондса на C++ та реалізація на Python (через `networkx.max_weight_matching`) будують паросполучення, обходячи вершини й ребра в різному порядку, тому іноді повертають **різні, але рівноцінні** максимальні паросполучення (той самий розмір, жодне не можна покращити).

Порівняння рузультатів і швидкодії роботи з реалізацією, яка використовує бібліотечні функції.

Для перевірки коректності та оцінки продуктивності власної реалізації алгоритму Едмондса (C++) я проводила порівняння з бібліотечною реалізацією в Python з використанням модуля **NetworkX** (`max_weight_matching` з параметром `maxcardinality=True`). Тестування виконувалося на випадково згенерованих графах з різною кількістю вершин (`n = 20 ... 300`) та фіксованою часткою заборонених вершин.


Співвідношення часу роботи Python до C++
Python / C++ (відношення часу)
Кількість вершин n
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
50
100
150
200
250
300


Порівняння часу роботи: C++ vs Python (networkx)
Максимальне паросполучення з забороненими вершинами
C++ (EdmondsBlossom)
Python + networkx
Середній час виконання, секунди
Кількість вершин n
0.000
0.005
0.010
0.015
0.020
0.025
0.030
0.035
50
100
150
200
250
300

Порівняння коректності результатів

У всіх тестових прикладах:

- розмір максимального паросполучення, обчислений власною реалізацією (C++) та бібліотечною реалізацією (Python + NetworkX), **завжди збігався**;
- жодного випадку, де один алгоритм знаходив більше ребер, ніж інший, не зафіксовано.

У деяких прикладах множини ребер могли відрізнятися за складом, однак:

- їх розмір був однаковим;
- кожне з них було коректним паросполученням;
- жодне з них не можна було покращити.

Це пояснюється тим, що в багатьох графах **максимальне паросполучення не є єдиним**: існує кілька різних множин ребер однакової максимальної потужності. Реалізації відрізняються порядком обходу вершин та ребер, тому можуть повертати різні, але рівноцінні розв'язки.

Висновок щодо коректності:

Власна реалізація алгоритму Едмондса дає такі самі результати за розміром паросполучення, як і бібліотечна реалізація, отже є коректною.

Порівняння швидкодії

Середні часи виконання (у секундах) для різних розмірів графа:

n	C++ (Edmonds)	Python + NetworkX
20	0.0238	0.0007
40	0.0208	0.0013
80	0.0216	0.0030
120	0.0277	0.0098
160	0.0234	0.0116
200	0.0221	0.0152
250	0.0250	0.0275

n	C++ (Edmonds)	Python + NetworkX
300	0.0269	0.0360

Аналіз графіків

1. Графік реального часу виконання

На графіку `benchmark_time.png` видно, що:

- для малих `n` Python працює швидше C++;
- починаючи приблизно з `n ≈ 250`, C++ стає помітно швидшим за Python;
- для великих графів різниця зростає на користь C++.

---

2. Графік співвідношення часу Python / C++

Графік `benchmark_ratio.png` показує відношення:

T (python)/ T (C++)

- при малих `n` це значення < 1, тобто Python виглядає швидшим;
- при зростанні `n` крива перетинає рівень 1;
- при `n = 300` Python працює приблизно в **1.3 раза повільніше**, ніж C++.

Чому на малих n Python швидше?

C++ реалізація в бенчмарку викликається через:

```
subprocess.run("./max_matching_forbidden.exe ...")
```

Тобто на кожен запуск включається:

- створення нового процесу,
- завантаження виконуваного файлу,
- ініціалізація середовища виконання.

Для малих графів обчислення займає дуже мало часу, і цей накладний час запуску процесу **домінує**, тому реальний час C++ виглядає більшим, ніж у Python.

Для великих графів:

- час власне виконання алгоритму починає домінувати,
- і тоді проявляється перевага реалізації на C++.

Висновки

1. **Коректність**

   Власна реалізація алгоритму Едмондса на C++ повністю збігається з бібліотечною реалізацією NetworkX за результатами (розмір максимального паросполучення).

2. **Швидкодія**

   - Для малих графів Python виглядає швидше через накладні витрати на запуск C++-процесу.
   - Для середніх і великих графів реалізація на C++ працює швидше.
   - При зростанні розміру графа перевага C++ збільшується.

3. **Загальний висновок**

   Реалізація на C++ є:

   - коректною,
   - масштабованішою,
   - придатнішою для великих графів у порівнянні з бібліотечною реалізацією на Python.